

Задание 3. Разработка системы учета посещаемости занятий студентами

Постановка задачи. Необходимо реализовать сервер учета посещаемости занятий студентами.

Требования к реализации. Система должна:

- Хранить списки студентов, групп студентов, преподавателей с возможностью добавить новый объект, отредактировать существующий, удалить объект получить объект по идентификатору, получить список всех объектов (список групп получаем весь, списки студентов – по группе, список преподавателей – весь, но постранично).
- Хранить список дисциплин с аналогичными возможностями.
- Преподаватель должен иметь возможность создать в системе занятие, указав для него дисциплину, группу студентов, для которых занятие проводилось, дату занятия и время (номер пары).
- Предполагается, что созданные занятия можно будет редактировать и удалять.
- Преподаватель должен иметь возможность указать, кто из студентов фактически присутствовал на занятии (с возможностью исправления ошибочных данных).
- Сервер должен предоставлять возможность получить занятие по его id (вместе с данными о посещаемости) и список занятий (без данных о посещаемости). Список занятий выдается за период (даты начала и конца указываются в запросе) постранично. Дополнительные и необязательные фильтры: по группе и преподавателю.
- Сервер должен быть реализован с использованием фреймворка Spring Boot, реляционной СУБД (PostgreSQL или MySQL), миграций Flyway.
- API должно быть построено с соблюдением архитектуры REST, в ответах должны быть адекватные HTTP статусы.
- Входные данные должны проходить валидацию.
- При невозможности обработать запрос сервер должен возвращать разумный ответ.
- Ответы сервера должны быть стандартизированы, т.е.
корректный ответ: { "success": true, "data": <данные ответа> },
некорректный ответ:
{
 "success": false,
 "errorCode": <код ошибки>,
 "errorMessage": <сообщение>,
 "details": <информация об ошибке>
}
Поле details представляет собой массив строк и может отсутствовать.
- Сервер должен контролировать консистентность данных. Например, нельзя создавать студента при несуществующей группе, нельзя удалять сущность при наличии не удаленных зависимых сущностей. Предполагается, что названия групп уникальны, ФИО студентов и преподавателей также уникальны.
- Создать и сохранить коллекции запросов в Postman. Запросы сгруппировать по сущностям.
- Должен быть git-репозиторий проекта с регулярными комитами.

- Персональные данные (логины, пароли и т.д.) не должны храниться в git – репозитории и внедряться в проект через переменные среды.

Требования, обязательные в реальной жизни, но необязательные в этом задании для троечников.

- Настроено логирование действий пользователя.
- Развернута система миграций для БД.
- Сервер должен поддерживать аутентификацию по логину /паролю и авторизацию на базе JWT (лучше на базе двух токенов: access и refresh).
- Должны быть реализованы три роли: студент, преподаватель, админ.
 - Студент видит список своей группы, списки преподавателей и дисциплин, данные о посещаемости своей группы.
 - Преподаватель видит все списки, создает, редактирует, удаляет занятие и данные о посещаемости.
 - Админ использует все API без ограничений.
- API документировано в Swagger.
- Код покрыт тестами.
- Контейнеризация проекта с использованием docker.

Этапы решения.

1. Спроектировать API.
2. Спроектировать структуру БД.
3. Реализовать API для сущностей студент и группа студентов.
4. Реализовать остальное API.